

Módulo 8

Técnicas de Visualização de Dados

1 Objetivos de Aprendizagem

Ao concluir este módulo, o estudante será capaz de:

- ✓ 1. Compreender os princípios fundamentais de visualização de dados segundo Tufte e a Gramática dos Gráficos de Wilkinson.
- ✓ 2. Selecionar o tipo de gráfico mais adequado para cada combinação de tipo de dado e objetivo comunicativo.
- ✓ 3. Implementar gráficos estatísticos fundamentais (distribuição, relação, comparação, composição) com Matplotlib e Seaborn.
- ✓ 4. Aplicar técnicas de redução de dimensionalidade (PCA, t-SNE, UMAP) para visualização de dados de alta dimensão.
- ✓ 5. Criar visualizações interativas com Plotly e construir dashboards analíticos básicos com Dash e Streamlit.
- ✓ 6. Visualizar resultados de modelos de mineração de dados: curvas ROC, matrizes de confusão, importância de atributos e gráficos SHAP.
- ✓ 7. Identificar e evitar armadilhas comuns de visualização (eixos truncados, distorção 3D, excesso de informação).
- ✓ 8. Aplicar boas práticas de acessibilidade em visualizações, incluindo paletas acessíveis a daltônicos e uso de texto alternativo.

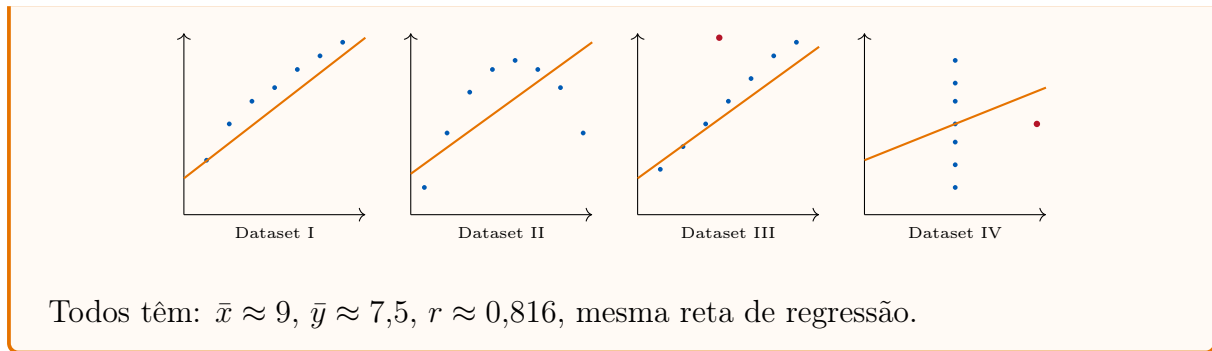
2 Princípios de Visualização de Dados

A visualização de dados é a representação gráfica de informação com o objetivo de comunicar padrões, tendências e anomalias de forma eficiente ao cérebro humano. Ela é simultaneamente uma ferramenta de **exploração** (EDA) e de **comunicação** (relatórios, dashboards).

2.1 Por que Visualizar?

💡 Quarteto de Anscombe (1973)

Francis Anscombe demonstrou que quatro conjuntos de dados podem ter médias, variâncias, correlações e linhas de regressão idênticas, mas padrões completamente diferentes quando visualizados. Isso ilustra que estatísticas resumidas **nunca substituem** a visualização dos dados.



Atributos pré-atentivos são propriedades visuais detectadas pelo sistema nervoso humano antes do processamento consciente (< 200 ms): cor, tamanho, orientação, posição, forma, movimento. Usá-los corretamente aumenta dramaticamente a eficiência da visualização.

2.2 Princípios de Tufte (1983)

Edward Tufte, no seminal *The Visual Display of Quantitative Information*, estabelece princípios que guiam o design de visualizações eficientes:

- **Data-ink ratio:** maximize a proporção de tinta usada para representar dados versus tinta decorativa. $\text{Data-ink ratio} = \frac{\text{tinta de dados}}{\text{tinta total}}$. Elimine grades desnecessárias, bordas, sombras.
- **Chartjunk:** evite elementos decorativos que não transmitem informação (padrões de preenchimento excessivos, imagens embutidas, perspectiva 3D desnecessária).
- **Lie Factor:** razão entre o efeito visual e o efeito real nos dados:

$$LF = \frac{\text{tamanho do efeito no gráfico}}{\text{tamanho do efeito nos dados}} \quad (1)$$

$LF = 1$ é ideal; desvios sistemáticos configuram manipulação visual.

- **Small multiples:** conjuntos de gráficos com a mesma escala e estrutura para facilitar comparação temporal ou entre grupos.
- **Densidade de dados:** maximize a quantidade de informação por unidade de área.

2.3 Grammar of Graphics (Wilkinson, 2005)

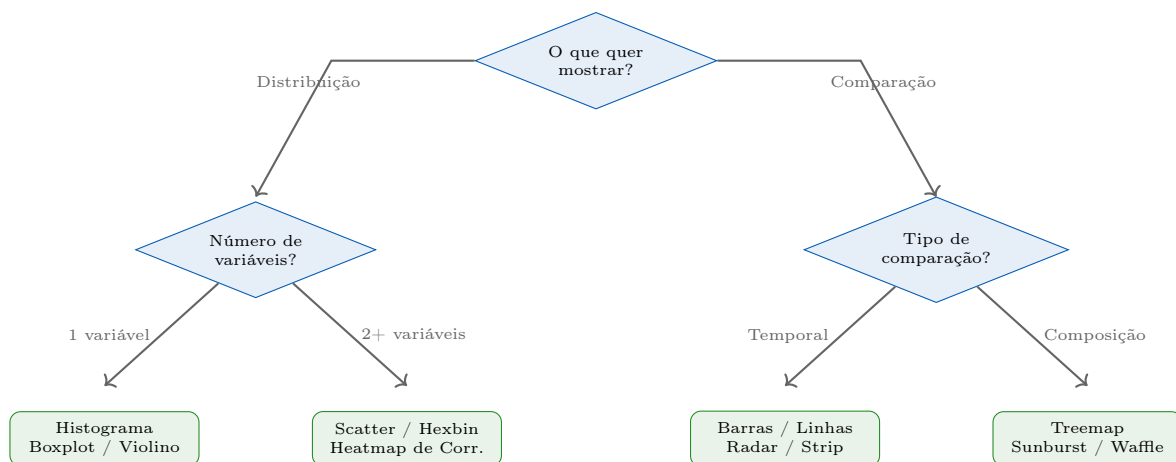
Leland Wilkinson formalizou a **Gramática dos Gráficos** como um sistema composicional com sete componentes:

1. **Data:** conjunto de dados subjacente.
2. **Aesthetics (aes):** mapeamento de variáveis a propriedades visuais (x, y, cor, tamanho, forma, opacidade).
3. **Geometry (geom):** tipo de elemento geométrico (ponto, linha, barra, polígono).

4. **Facets:** subdivisão em painéis por variável categórica.
5. **Statistics (stat):** transformações estatísticas (binning, suavização, contagem).
6. **Coordinates (coord):** sistema de coordenadas (cartesiano, polar, mapa).
7. **Themes:** aparência visual não-data (fonte, cor de fundo, grade).

Implementações: **ggplot2** (R, Wickham 2010), **plotnine** (Python), **Plotly** (implementa parcialmente a gramática via `go.Figure`).

2.4 Escolha do Tipo de Gráfico



3 Gráficos Estatísticos Fundamentais

3.1 Distribuição

- **Histograma:** divide dados em bins e conta frequência. Use para exploração inicial e detecção de bimodalidade. Cuidado com número de bins.
- **KDE (Kernel Density Estimate):** versão suavizada do histograma, sem a arbitrariedade do binning.
- **Boxplot:** mediana, IQR, whiskers ($\pm 1,5 \times IQR$) e outliers. Excelente para comparação de grupos.
- **Violin Plot:** combina boxplot com KDE; mostra a distribuição completa, especialmente útil quando bimodal.
- **Q-Q Plot:** compara a distribuição observada com uma teórica (e.g., normal); pontos na diagonal indicam ajuste.

Tabela 1: Guia de seleção de tipos de gráfico

Tipo de Gráfico	Tipo de Dado	Caso de Uso	Exemplo	Ferramentas
Histograma	Numérico	Distribuição uni-variada	Distribuição de idades	matplotlib, seaborn
Boxplot	Num. por categ.	Comparar distribuições	Notas por turma	seaborn, plotly
Scatter Plot	2 Numéricos	Relação/correlação	Renda vs consumo	matplotlib, plotly
Heatmap	Matriz	Correlações, matrizes	Corr. de features	seaborn, plotly
Barras	Categ. + Num.	Comparação de grupos	Vendas por região	matplotlib, plotly
Linha	Temporal	Séries temporais	Acurácia por época	matplotlib, plotly
Treemap	Hierárquico	Composição aninhada	Receita por produto	plotly, squarify
Violino	Num. por categ.	Distribuição + densidade	Salário por cargo	seaborn
Choropleth	Geoespacial	Distribuição geográfica	IDH por estado	plotly, folium
Parallel Coord.	Multivariado	Alta dimensão	Comparar clusters	plotly

Listing 1: Gráficos de distribuição com seaborn

```

1 import seaborn as sns, matplotlib.pyplot as plt
2
3 fig, axes = plt.subplots(1, 3, figsize=(14, 4))
4 sns.histplot(data=df, x='feature', kde=True, ax=axes[0])
5 axes[0].set_title('Histograma + KDE')
6
7 sns.boxplot(data=df, x='category', y='feature', ax=axes[1])
8 axes[1].set_title('Boxplot por Grupo')
9
10 sns.violinplot(data=df, x='category', y='feature',
11               inner='quartile', ax=axes[2])
12 axes[2].set_title('Violin Plot')
13 plt.tight_layout(); plt.show()

```

3.2 Relação entre Variáveis

- **Scatter Plot:** relação entre duas variáveis contínuas. Adicione cor/tamanho para terceira/quarta dimensão.
- **Bubble Chart:** scatter com tamanho proporcional a uma terceira variável quantitativa.
- **Hexbin:** versão do scatter para grandes datasets (evita overplotting com contagem

por célula hexagonal).

- **Joint Plot:** scatter central + distribuições marginais. Seaborn `JointGrid`.
- **Correlation Heatmap:** matriz de correlações com codificação de cor; use máscara para triângulo superior.
- **Pair Plot (SPLOM):** todos os scatter plots par a par em uma grade. Ideal para EDA inicial.

3.3 Comparação

- **Gráfico de barras agrupado:** compara múltiplos grupos por categoria. Evite mais de 5 grupos.
- **Gráfico de barras empilhado:** mostra contribuição de subgrupos ao total.
- **Lollipop chart:** alternativa mais limpa ao gráfico de barras, alta data-ink ratio.
- **Radar / Spider Chart:** compara perfis multivariados em múltiplos eixos radiais; bom para comparação de clusters.
- **Time Series Line Plot:** tendências temporais com múltiplas séries. Adicione bandas de confiança.
- **Strip / Swarm Plot:** mostra todos os pontos individuais, complementa o boxplot.

3.4 Composição

- **Gráfico de pizza:** use com moderação – apenas com 2–4 categorias, quando a proporção relativa ao todo é o foco. **Nunca use 3D.**
- **Treemap:** área proporcional ao valor; excelente para dados hierárquicos.
- **Sunburst:** versão radial do treemap para hierarquias multinível.
- **Waffle Chart:** grade de quadrados onde cada quadrado representa percentual; mais preciso visualmente que pizza.

3.5 Geoespacial

- **Choropleth Maps:** polígonos geográficos coloridos por valor (e.g., IDH por estado). Implementação com Plotly ou Folium.
- **Scatter Geo:** pontos em mapa para dados com coordenadas (lat/lon).
- **Folium/Leaflet:** mapas interativos no browser com layers, clusters e tooltips.
- **Cartopy:** projeções cartográficas científicas integradas ao Matplotlib.

4 Visualização de Alta Dimensão

Datasets de mineração de dados frequentemente possuem dezenas ou centenas de features. A visualização direta é impossível em $d > 3$, tornando necessárias técnicas de redução de dimensionalidade para projeção em 2D ou 3D.

4.1 Redução para 2D/3D

4.1.1 PCA (Principal Component Analysis)

A PCA projeta os dados nas direções de máxima variância:

$$\mathbf{Z} = \mathbf{X}\mathbf{W}, \quad \text{onde } \mathbf{W} = \text{eigenvectors de } \mathbf{X}^T\mathbf{X} \quad (2)$$

O **PCA biplot** mostra simultaneamente as observações (pontos) e os loadings das variáveis originais (vetores), revelando quais variáveis mais contribuem para cada componente principal.

4.1.2 t-SNE (van der Maaten & Hinton, 2008)

t-SNE minimiza a divergência KL entre a distribuição de pares no espaço original (Gaussiana) e no espaço reduzido (t-Student):

$$KL(P\|Q) = \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}} \quad (3)$$

Características: não-linear, estocástico, excelente para visualizar clusters. **Parâmetro crítico:** perplexidade (5–50). **Limitação:** lento para $n > 10.000$, não preserva estrutura global.

4.1.3 UMAP (McInnes et al., 2018)

UMAP baseia-se em geometria Riemanniana e teoria de topologia fuzzy. Preserva tanto estrutura local quanto global, é mais rápido que t-SNE e permite transformar novos pontos (fit/transform).

Tabela 2: Comparação: PCA vs t-SNE vs UMAP

Critério	PCA	t-SNE	UMAP
Tipo	Linear	Não-linear	Não-linear
Velocidade	Muito rápido	Lento ($O(n^2)$)	Rápido ($O(n \log n)$)
Estrutura global	Boa	Ruim	Boa
Estrutura local	Moderada	Excelente	Excelente
Determinismo	Sim	Não	Não (seed fix)
Parâmetros	Nenhum	Perplexidade	n_neighbors, min_dist
Transform novos	Sim	Não	Sim

Listing 2: Comparação PCA, t-SNE, UMAP para visualização

```
1 from sklearn.manifold import TSNE
2 from sklearn.decomposition import PCA
3 import umap
4
5 # PCA
6 pca = PCA(n_components=2)
7 X_pca = pca.fit_transform(X_scaled)
8
9 # t-SNE
10 tsne = TSNE(n_components=2, perplexity=30, random_state=42)
11 X_tsne = tsne.fit_transform(X_scaled)
12
13 # UMAP
14 reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=42)
15 X_umap = reducer.fit_transform(X_scaled)
```

4.2 Parallel Coordinates

Cada variável é representada como um eixo vertical paralelo, e cada observação é uma linha poligonal que cruza todos os eixos. Permite identificar clusters, outliers e correlações em dados de alta dimensão. Plotly oferece `go.Parcoords` com interatividade (brushing por eixo).

4.3 Matriz de Dispersão (SPLOM)

Todos os scatter plots pairwise em uma grade $d \times d$. Diagonal pode mostrar histogramas ou KDE. Codificação por cor revela separação de classes. `plotly.express.scatter_matrix()` gera SPLOM interativo com hover e seleção.

5 Visualização Interativa

5.1 Plotly

Plotly é a biblioteca de visualização interativa mais popular para Python, gerando gráficos baseados em D3.js:

- **Plotly Express:** API de alto nível para gráficos com uma linha de código. `px.scatter()`, `px.bar()`, `px.choropleth()`.
- **go.Figure:** API de baixo nível para layouts customizados, múltiplos eixos, anotações.
- **Animações:** parâmetro `animation_frame` em Express ou `frames` em Figure.
- **Subplots:** `make_subplots(rows, cols)` com compartilhamento de eixos.

Listing 3: Dashboard interativo com Plotly subplots

```

1 import plotly.graph_objects as go
2 from plotly.subplots import make_subplots
3
4 fig = make_subplots(rows=2, cols=2,
5     subplot_titles=["ROC Curve", "Confusion Matrix",
6         "Feature Importance", "Learning Curve"])
7
8 fig.add_trace(go.Scatter(x=fpr, y=tp, name=f"AUC={auc:.2f}"),
9     row=1, col=1)
10 fig.add_trace(go.Heatmap(z=cm, colorscale="Blues"), row=1, col=2)
11 fig.add_trace(go.Bar(x=importances, y=features, orientation='h'),
12     row=2, col=1)
13 fig.update_layout(title="Dashboard de Avaliacao de Modelo",
14     height=700)
15 fig.show()

```

5.2 Bokeh

Bokeh oferece interatividade server-side avançada:

- **Linked plots (brushing):** seleção em um gráfico reflete em todos os outros conectados.
- **Bokeh Server:** callbacks Python ativados por interações do usuário.
- **Hover tools:** tooltips customizados com HTML.

5.3 Dash

Dash (Plotly) é um framework para construção de aplicações analíticas web em Python puro:

Listing 4: Aplicação Dash básica com callback

```

1 import dash
2 from dash import dcc, html, Input, Output
3 import plotly.express as px
4
5 app = dash.Dash(__name__)
6 app.layout = html.Div([
7     dcc.Dropdown(id='x-axis', options=['feature1', 'feature2'], value='feature1'),
8     dcc.Graph(id='scatter-plot')
9 ])
10
11 @app.callback(Output('scatter-plot', 'figure'),
12     Input('x-axis', 'value'))
13 def update_figure(x_col):
14     return px.scatter(df, x=x_col, y='target', color='cluster')
15
16 if __name__ == '__main__':
17     app.run(debug=True)

```


5.4 Streamlit

Streamlit permite criar dashboards analíticos em minutos:

Listing 5: Dashboard Streamlit para resultados de DM

```
1 import streamlit as st
2 import plotly.express as px
3
4 st.title("Dashboard: Mineracao de Dados - UEPA")
5 with st.sidebar:
6     k = st.slider("Numero de clusters", 2, 10, 4)
7
8 tab1, tab2 = st.tabs(["Clustering", "Classificacao"])
9 with tab1:
10     st.plotly_chart(px.scatter(df_tsne, x='x', y='y', color=f'cluster_{k}'))
11 with tab2:
12     st.dataframe(df_metrics, use_container_width=True)
```

6 Visualização de Resultados de Mineração de Dados

6.1 Avaliação de Modelos

- **Curva ROC:** True Positive Rate vs False Positive Rate para diferentes thresholds. AUC-ROC mede a área sob a curva; AUC = 1 é perfeito, 0,5 é aleatório.
- **Curva Precision-Recall:** preferível à ROC em datasets muito desbalanceados. AUC-PR é mais informativa quando a classe positiva é rara.
- **Matriz de Confusão (Heatmap):** visualização de TP, FP, FN, TN. Normalize por linha para mostrar taxas de acerto por classe.
- **Curvas de Aprendizagem:** desempenho no treino e validação em função do tamanho do dataset. Diagnostica underfitting (ambas baixas) e overfitting (treino alta, validação baixa).

6.2 Interpretabilidade Visual

6.2.1 SHAP (SHapley Additive exPlanations)

SHAP (Lundberg & Lee, 2017) atribui a cada feature a contribuição marginal para a predição individual:

- **Summary Plot (Beeswarm):** todos os pontos de todos os exemplos; cor = valor da feature, eixo x = impacto SHAP. Revela distribuição de importância.
- **Bar Summary:** média do valor absoluto SHAP por feature; ranking global.
- **Waterfall Plot:** para uma predição individual; detalha contribuição de cada feature a partir do valor base.

- **Dependence Plot:** SHAP value vs valor da feature, colorido por feature de interação.

Listing 6: Visualizações SHAP para interpretabilidade

```
1 import shap
2
3 explainer = shap.TreeExplainer(model)
4 shap_values = explainer.shap_values(X_test)
5
6 # Summary beeswarm
7 shap.summary_plot(shap_values, X_test, feature_names=feature_names)
8
9 # Waterfall para observacao 0
10 shap.waterfall_plot(shap.Explanation(
11     values=shap_values[0],
12     base_values=explainer.expected_value,
13     data=X_test.iloc[0],
14     feature_names=feature_names))
```

6.2.2 LIME

LIME (Ribeiro et al., 2016) explica predições individuais aproximando o modelo com um modelo linear local. A visualização mostra features com contribuição positiva (verde) e negativa (vermelho) para a predição.

6.3 Clustering Visualization

- **t-SNE / UMAP colorido por cluster:** visualização 2D dos clusters descobertos.
- **Dendrograma:** hierarquia de fusões no clustering aglomerativo.
- **Silhouette Plot:** gráfico de coeficientes de silhueta por amostra e cluster. Coeficiente $s_i \in [-1, 1]$; $s_i > 0$ bem clustered, $s_i < 0$ mal clustered.
- **Elbow Curve:** inércia vs k ; o “cotovelo” sugere o k ótimo.
- **Radar Chart dos Centroides:** perfil multivariado de cada cluster em gráfico radial.

6.4 Associação

- **Grafo de regras:** nós são itemsets, arestas são regras; espessura = confiança, cor = lift.
- **Heatmap de co-ocorrência:** frequência de pares de itens.
- **Bubble Chart:** suporte (eixo x) vs confiança (eixo y), tamanho da bolha = lift. Permite identificar regras interessantes visualmente.

7 Dashboards Analíticos

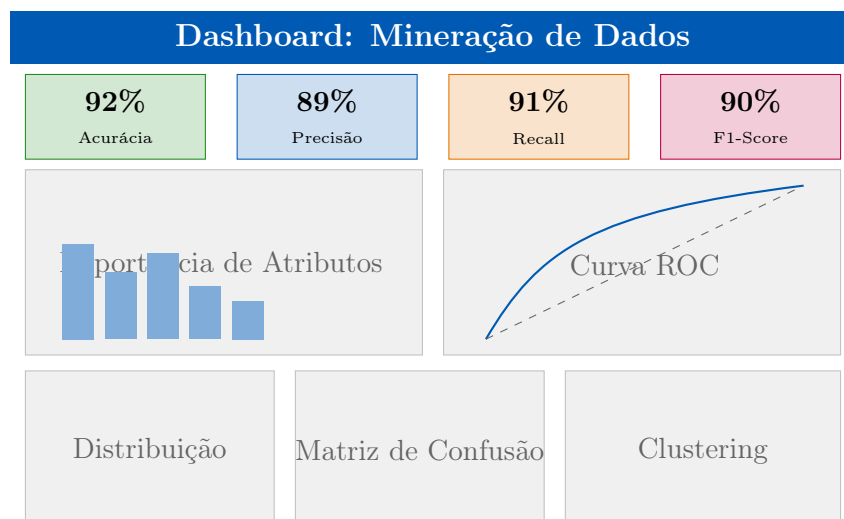
Dashboard Analítico

Um dashboard é uma interface visual que consolida múltiplos indicadores e gráficos em uma única tela, permitindo monitoramento de KPIs e análise exploratória em tempo real.

Princípios de design de dashboards:

1. **Hierarquia visual:** os itens mais importantes devem ser maiores e mais proeminentes (KPI cards no topo).
2. **Alinhamento:** alinhe elementos em uma grade para criar ordem visual.
3. **Proximidade:** agrupe elementos relacionados visualmente.
4. **Contraste:** diferencie seções com cor ou espaço.
5. **Drill-down:** permita ao usuário navegar do macro para o micro.
6. **Mobile responsiveness:** layouts que se adaptam a telas menores.

Mockup de Dashboard Analítico para DM



8 Boas Práticas e Armadilhas Comuns

8.1 Erros Comuns

⚠ Principais Armadilhas na Visualização

- **Eixo y truncado:** cortar o eixo y para exagerar diferenças pequenas. Lie Factor elevado. Sempre comece o eixo y em zero para gráficos de barra.
- **Gráfico de pizza 3D:** perspectiva deforma as áreas, impossibilitando comparação precisa.
- **Excesso de cores:** mais de 7 categorias em uma legenda de cor torna o gráfico ilegível. Agrupe categorias menores em “Outros”.
- **Ausência de unidades e rótulos:** eixos sem título e unidade tornam o gráfico ininterpretável.
- **Gráficos de duplo eixo:** dois eixos y diferentes permitem manipulação visual da correlação aparente entre séries.
- **Overplotting:** em scatter com muitos pontos, use transparência (**alpha**) ou hexbin.
- **Rainbow colormap:** *jet/rainbow* não são perceptualmente uniformes; use *viridis* ou *plasma*.

8.2 Acessibilidade

- **Daltonismo:** cerca de 8% dos homens têm dificuldade de distinguir vermelho e verde. Use paletas acessíveis: **ColorBrewer**, **viridis**, **cividis**, **Wong** (8 cores acessíveis).
- **Alto contraste:** razão mínima de 4,5:1 entre texto e fundo (WCAG 2.1 nível AA).
- **Alt text:** para imagens em relatórios web/PDF, descreva o padrão principal do gráfico.
- **Não dependência de cor exclusiva:** use combinação de cor + forma + padrão para codificar categorias.

9 Estado da Arte (2020–2024)

🔧 Visualização Exploratória Automática (Auto-EDA)

Ferramentas de EDA automatizada eliminam a necessidade de escrever código para a exploração inicial de datasets. **ydata-profiling** (ex-Pandas Profiling) gera relatórios HTML completos com estatísticas, correlações, valores ausentes e alertas de qualidade. **Sweetviz** compara dois datasets (treino vs teste) destacando diferenças de distribuição. **AutoViz** gera automaticamente os gráficos mais relevantes.

lux recomenda visualizações contextuais no Jupyter Notebook. **D-Tale** oferece exploração interativa com manipulação point-and-click.

Visualização Explicável (XAI Viz)

SHAP tornou-se o padrão da indústria para explicabilidade de modelos de ML, com plots integrados ao scikit-learn e XGBoost. **DiCE** (Diverse Counterfactual Explanations) gera exemplos contrafactuais visuais: “o que precisaria mudar para inverter a predição?”. O **What-if Tool** do Google permite exploração visual interativa de modelos no TensorBoard. **Alibi** oferece explicações anchor e contrafactuais. A tendência é integrar XAI diretamente nos dashboards de produção para auditoria contínua.

BI Moderno e DataViz

Apache Superset (open-source) compete com Tableau e Power BI para dashboards corporativos. **Metabase** simplifica o acesso de usuários não técnicos a dados. **Grafana** tornou-se padrão para dados de séries temporais e monitoramento de sistemas. **Observable Plot** (JavaScript, 2022) é a evolução do D3.js, mais acessível. **Evidence** e **Rill** emergem como ferramentas de BI baseadas em SQL com output estático. Vibe-coded dashboards com LLMs (Text-to-Viz) começam a democratizar a criação de visualizações para usuários não programadores.

10 Exercícios

1. **[Quarteto de Anscombe]** Crie manualmente os quatro datasets do Quarteto de Anscombe usando NumPy. Calcule e imprima as estatísticas de cada um (média, variância, correlação, coeficientes de regressão linear). Em seguida, plote todos em um grid 2×2 com matplotlib. O que esse exercício ensina sobre a necessidade de visualização?
2. **[Design de Dashboard]** Para o projeto de DM do módulo anterior (classificação ou clustering), projete um dashboard analítico no papel (wireframe) com: (a) 4 KPI cards no topo, (b) gráfico de avaliação do modelo, (c) visualização dos dados com PCA/t-SNE, (d) importância de atributos. Implemente o wireframe como figura Plotly `make_subplots`.
3. **[Plotly Interativo]** Crie um scatter plot interativo com Plotly Express usando o dataset Iris: (a) cores por espécie, (b) tamanho dos pontos proporcional ao comprimento da pétala, (c) hover customizado mostrando todas as 4 features, (d) botões de animação para alternar entre pares de features.
4. **[Visualização SHAP]** Treine um XGBoost no dataset de câncer de mama (sklearn). Gere: (a) beeswarm summary plot, (b) bar summary plot, (c) waterfall plot para 3 exemplos (um verdadeiro positivo, um falso positivo, um verdadeiro negativo). Interprete cada plot em 3 frases.

5. **[t-SNE vs UMAP]** Carregue o dataset MNIST (use `fetch_openml('mnist_784')` ou uma amostra de 5000 pontos). Aplique PCA (primeiro reduzir a 50D), depois t-SNE e UMAP. Compare visualmente a separação das 10 classes. Meça o tempo de execução de cada método. Qual preserva melhor a estrutura global?
6. **[Auto-EDA Report]** Carregue qualquer dataset com pelo menos 10 colunas e 500 linhas. Execute `ydata_profiling.ProfileReport` e salve o relatório HTML. Identifique: (a) 3 problemas de qualidade de dados detectados automaticamente, (b) 2 correlações interessantes entre variáveis, (c) distribuições problemáticas (multimodal, skewed, outliers).
7. **[Acessibilidade]** Crie o mesmo gráfico de barras em três versões: (a) versão padrão com cores arbitrárias, (b) versão com paleta ColorBrewer acessível a daltônicos, (c) versão com hachuras (patterns) além de cores. Use o simulador de daltonismo Coblis ou `matplotlib.colors` para verificar a acessibilidade. Documente as diferenças.

11 Referências

Referências

- [1] Tufte, E. R. (1983). *The Visual Display of Quantitative Information*. Graphics Press.
- [2] Wilkinson, L. (2005). *The Grammar of Graphics* (2nd ed.). Springer.
- [3] van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.
- [4] McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv:1802.03426*.
- [5] Wickham, H. (2010). A layered grammar of graphics. *Journal of Computational and Graphical Statistics*, 19(1), 3–28.
- [6] Cairo, A. (2016). *The Truthful Art: Data, Charts, and Maps for Communication*. New Riders.
- [7] Munzner, T. (2014). *Visualization Analysis and Design*. CRC Press.
- [8] Few, S. (2012). *Show Me the Numbers: Designing Tables and Graphs to Enlighten* (2nd ed.). Analytics Press.
- [9] Lundberg, S. M., & Lee, S. I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30* (NIPS 2017).
- [10] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. In *Proceedings of KDD 2016* (pp. 1135–1144).
- [11] Anscombe, F. J. (1973). Graphs in statistical analysis. *The American Statistician*, 27(1), 17–21.